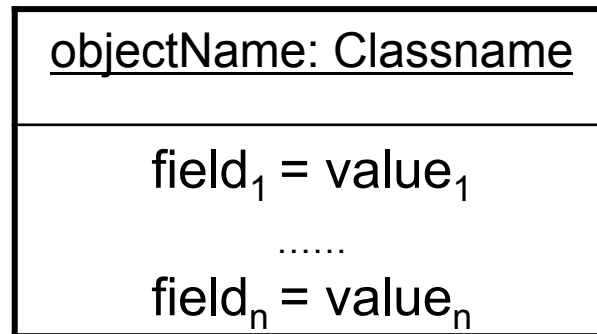


Behavioral Models

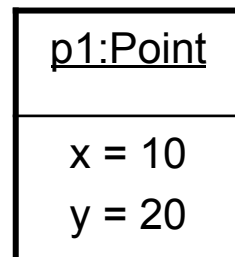
Notation for Objects

- Object: Rectangular box with one or two compartments
- Object name: object name/role name:class name

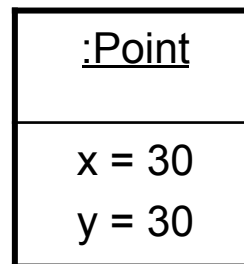


The top compartment shows the name of the object and its class.

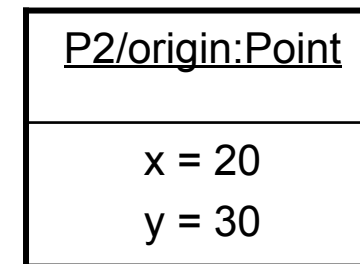
The bottom compartment contains a list of the fields and their values.



named



anonymous



**named
with role**

Object = Identity + State + Behavior

- Identity
 - Distinguishes an object from all other objects
- State
 - Consists of a set of *attributes* (or *fields*), which have names, types, and values
- Behavior
 - Defined by the set of *operations* (or *methods*) that may operate on the object
 - Each method has a name, a type, and a value, where
 - The **type** consists of the return type and the list of parameter types of the method, often called *signature*.
 - The **value** is the implementation of the method often expressed as a sequence of statements, in languages like Java and C++

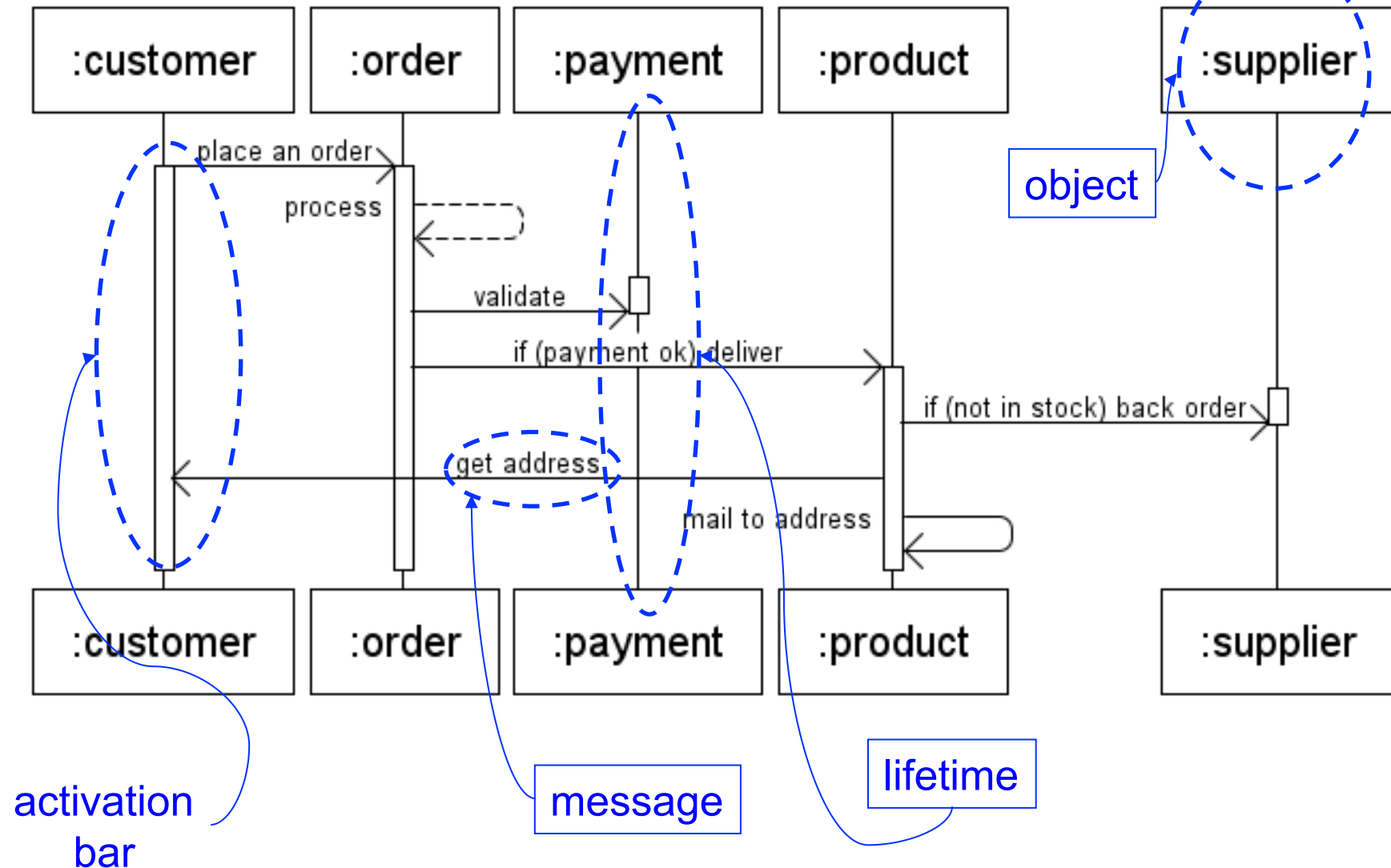
Interaction diagrams

- A use case diagram presents an **outside view** of the system
- The **inside view** of a system is shown by **interaction diagrams**
- Interaction diagrams describe how a use case is realized in terms of interacting objects
- Two types of interaction diagrams
 - **Sequence** diagrams
 - **Collaboration** (Communication) diagrams

Sequence diagram: main entities

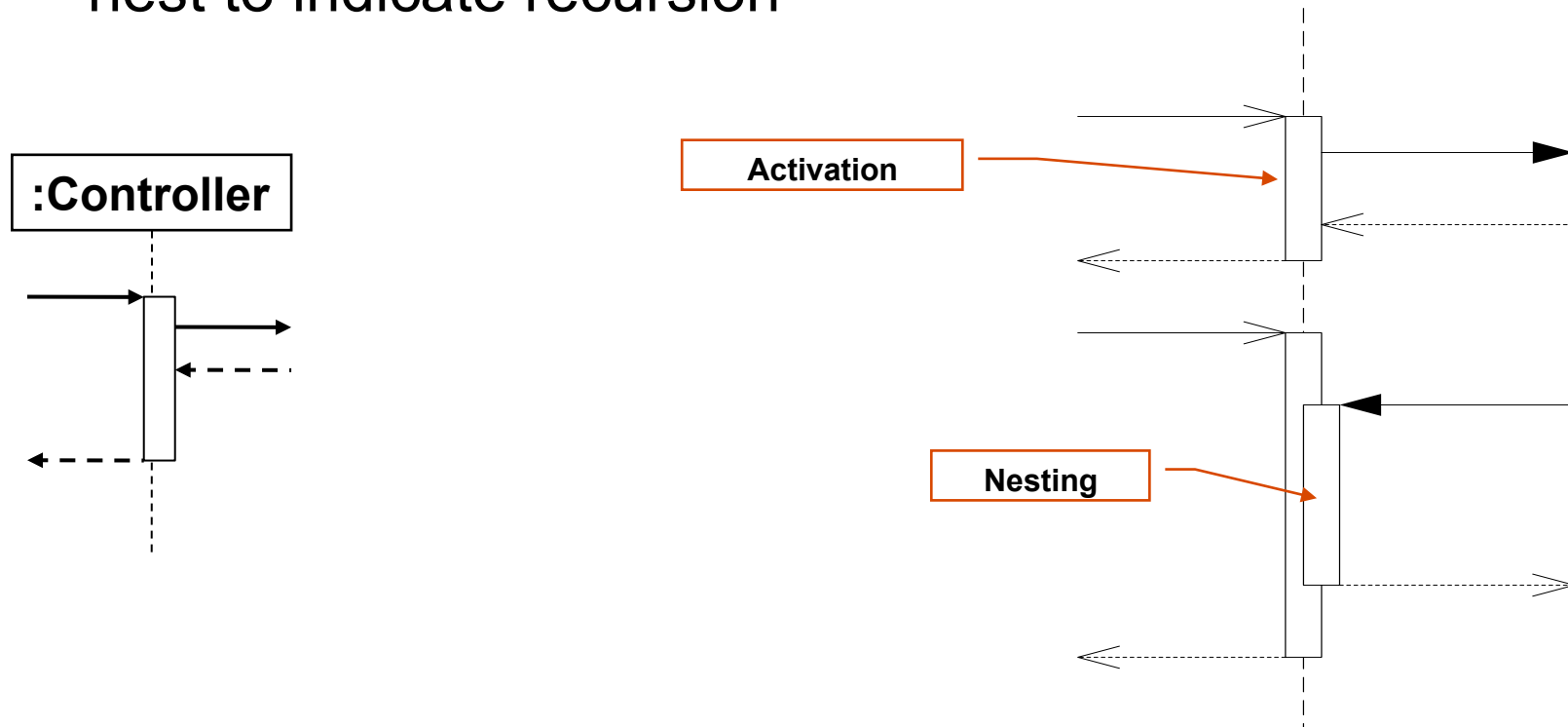
- **participant**: an object that acts in the sequence diagram
- **message**: communication between participant objects
- the **axes** in a sequence diagram:
 - horizontal: which object/participant is acting
 - vertical: time (down -> forward in time)

Sequence diagram: elements



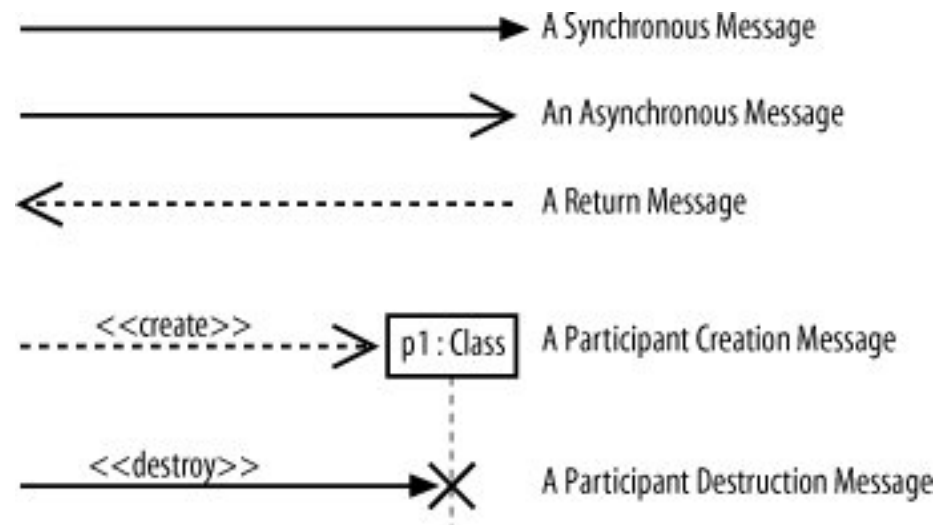
Indicating method calls

- **activation**: thick box over object's life line
 - The object is running its code, or it is waiting for another object's method to finish
 - nest to indicate recursion



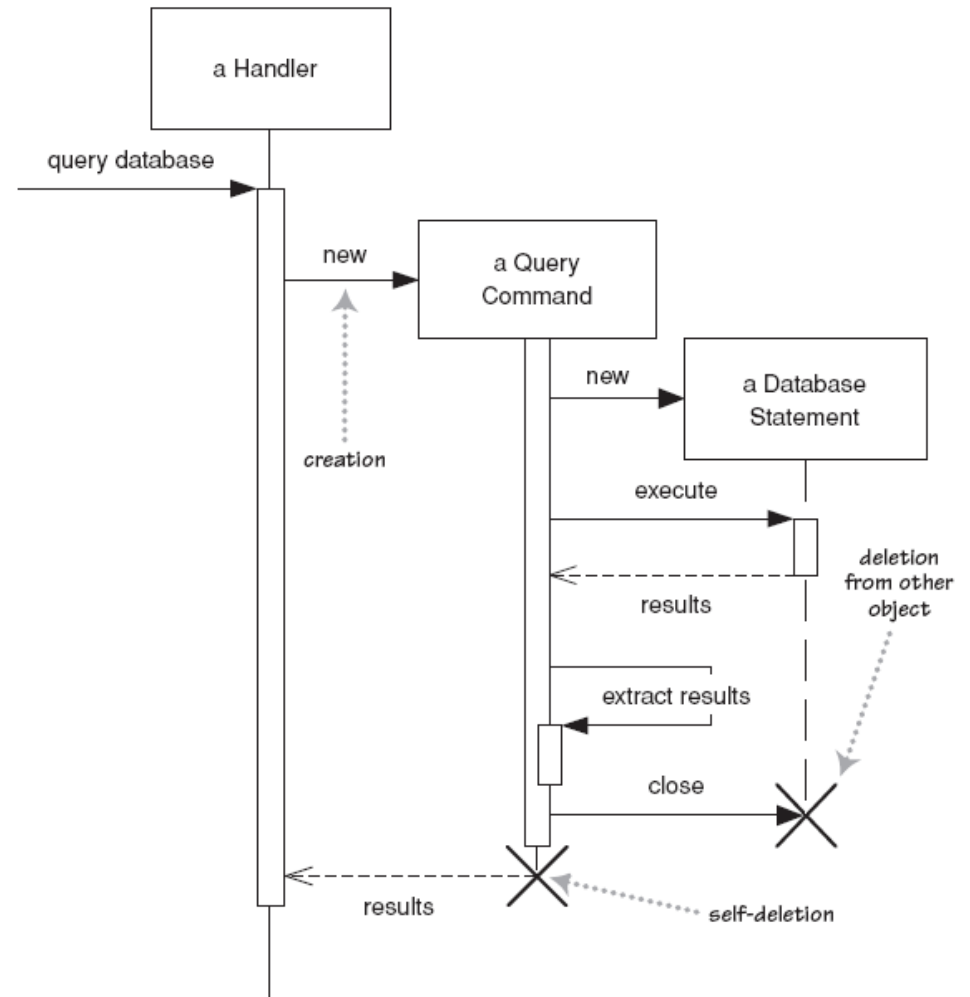
Arrows in a sequence diagram

- A sequence diagram describes sequences of method calls among objects
- There are several types of method calls

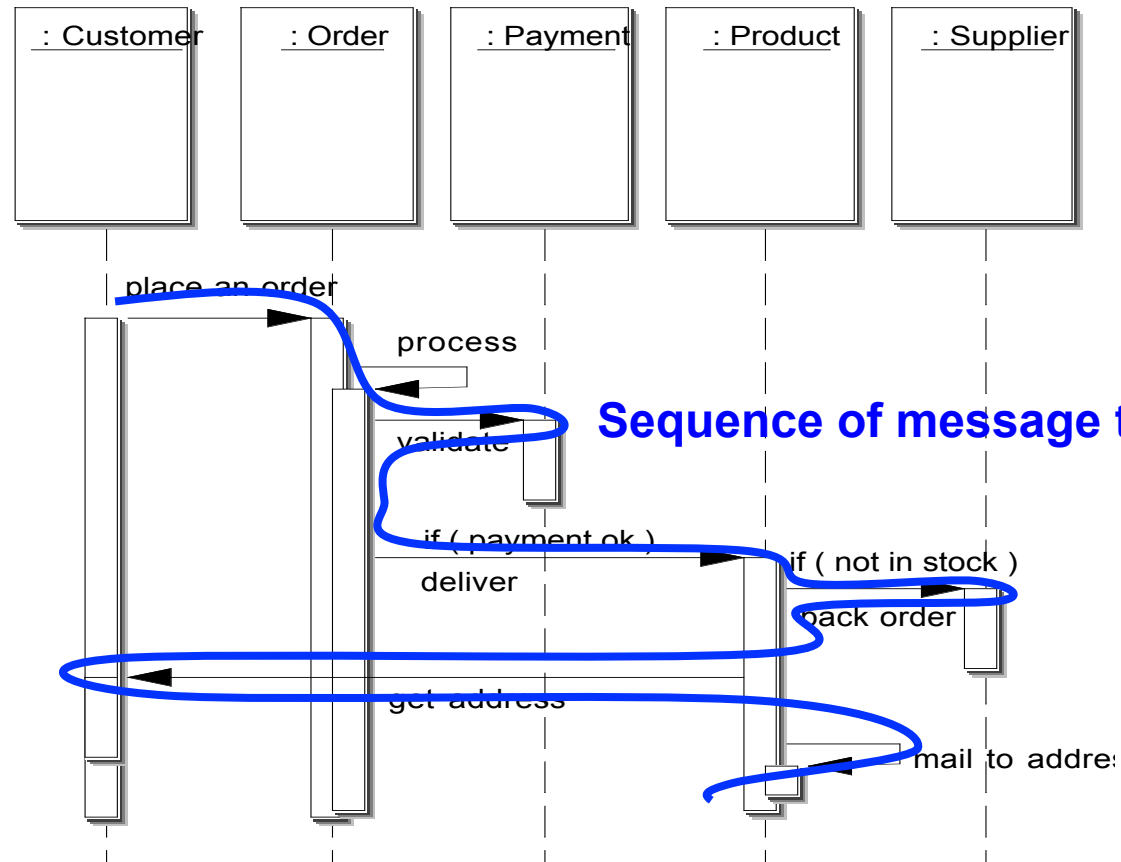


Lifetime of objects

- creation: arrow with 'new' label
 - an object created after the start of the scenario appears lower than the others
- deletion: an X at bottom of object's lifeline
 - Java does not explicitly delete objects; they fall out of scope and are garbage-collected



Sequence diagram: flow



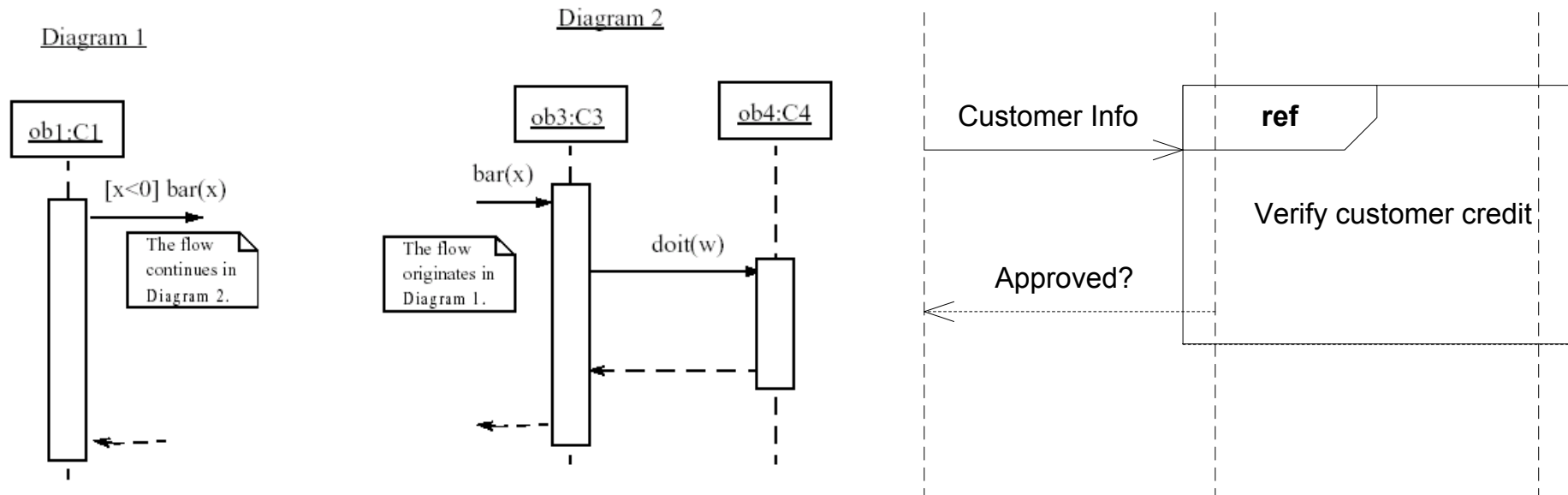
Sequence of message transmissions

Two Important SD elements

- Interaction Occurrence
 - Reusable interactions
 - Using ref element in the sequence diagram
 - To accommodate long sequence diagrams by invoking a SD from another SD
- Combined Fragments – for selection and loops in SD
 - Alt (equivalent of if)
 - Opt (equivalent of if-else)
 - Loop (equivalent of loop)

Linking sequence diagrams

- if a sequence diagram is too large or refers to another diagram, indicate it with either:
 - an unfinished arrow and a comment
 - a “**ref**” frame that names the other diagram

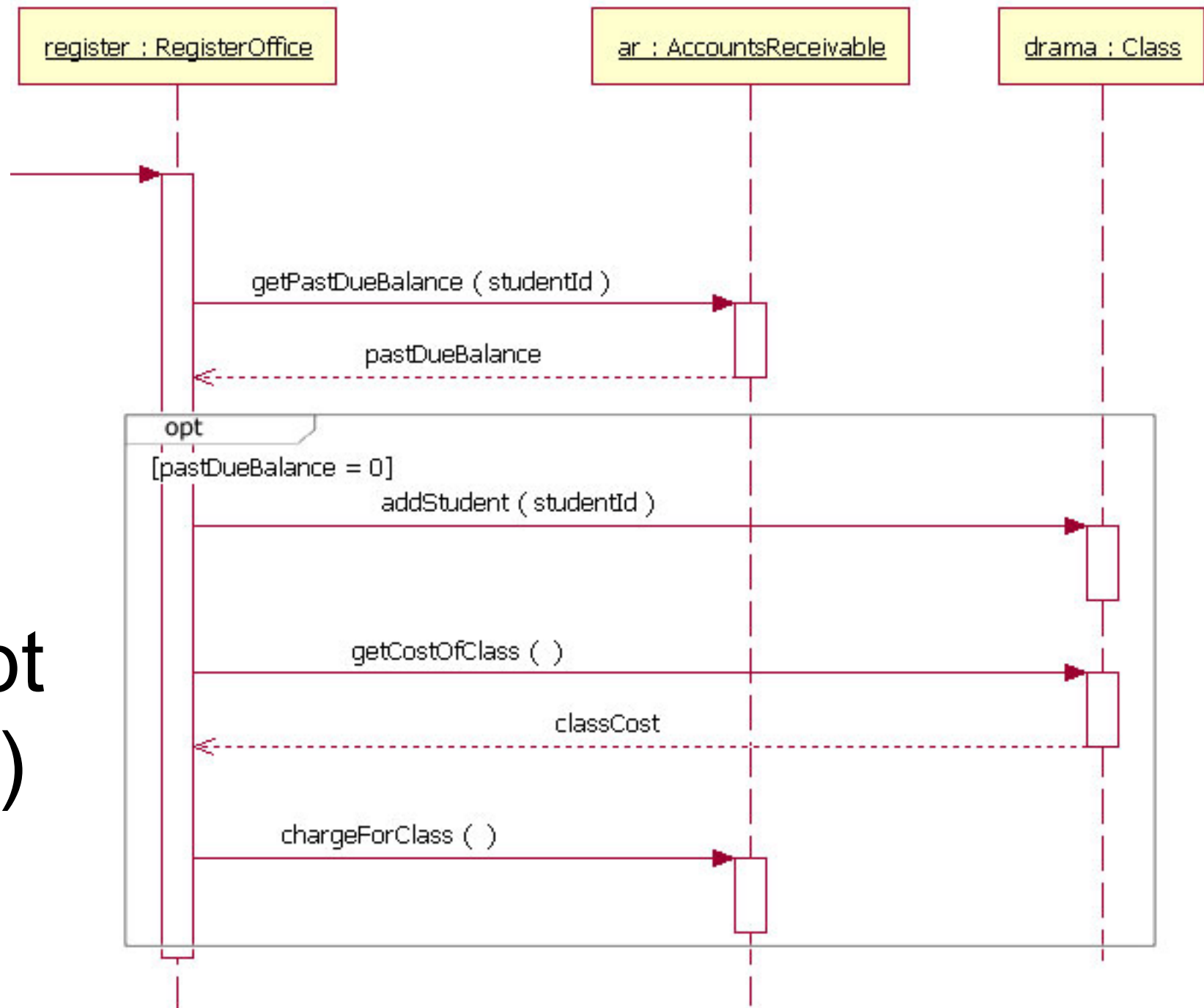


Conditionals and loops

- frame: box around part of a sequence diagram to indicate selection or loop
 - if : (`opt`) [`condition`]
 - if-else: (`alt`) [`condition`] else [`condition`]
 - loop: (`loop`) [`condition to loop over`]

Examples: see the next three slides

Opt
(if)



Alt (if-else)

